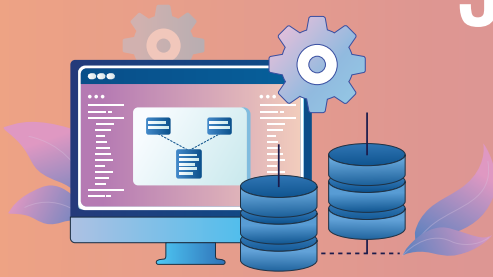
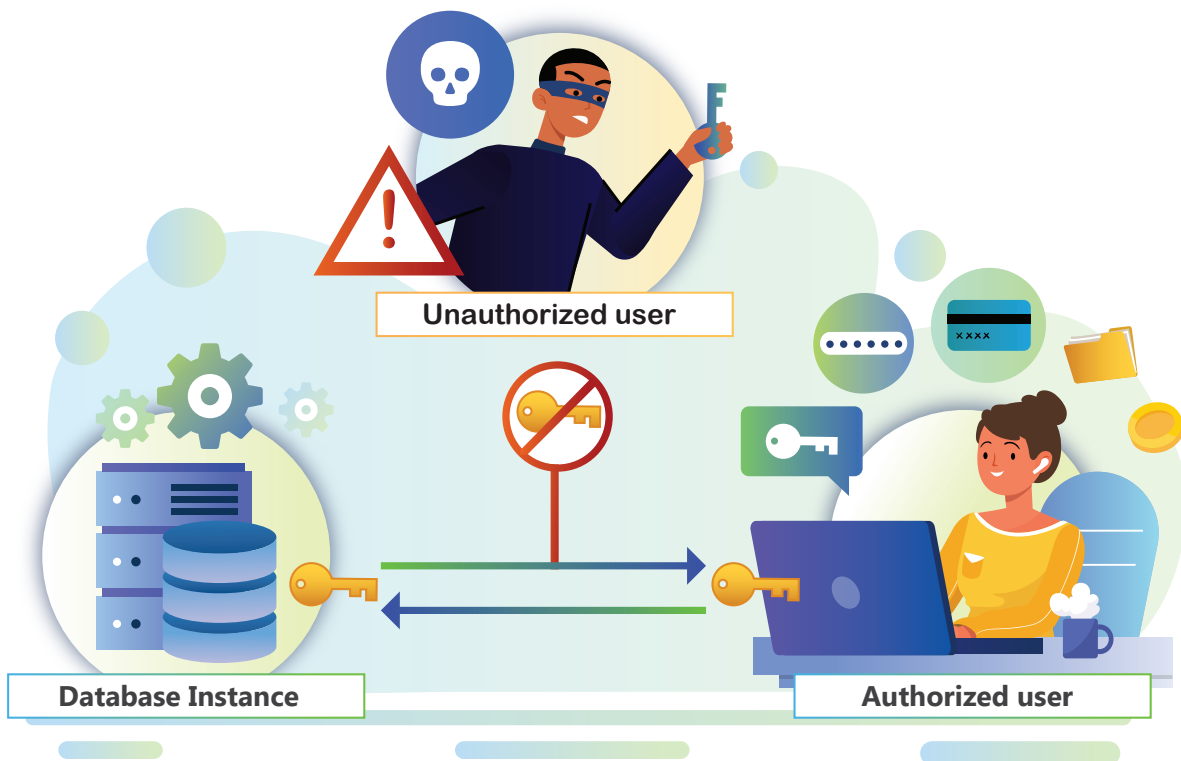


Joins & Database Security



05

Every day, enormous data is generated through shopping, social media, and banking. Databases organise, retrieve, and secure this information, making them essential in modern technology. Let us explore how they structure and protect data efficiently.



Databases store large amounts of information, such as student records, hospital data, and online transactions. They help in organising and retrieving data efficiently, making tasks faster and more accurate.

Consider a scenario where a school stores students and course details in different tables. If a teacher needs to check which student is enrolled in which course, they must search multiple tables separately. This process is time-consuming and inefficient. Joins help retrieve related data from multiple tables efficiently.

Recall:

Before we proceed, let us briefly review the key concepts discussed in the previous lesson:

- **Primary Key:** A unique field that identifies each record in a table. It cannot contain NULL values, and no two records can have the same primary key.
- **Foreign Key:** A field in one table that refers to the primary key of another table. It creates a link or establishes a relation between two or more tables.
- **Joins:** A technique in RDBMS used to retrieve data from multiple tables by establishing a connection between them using primary and foreign keys.

This lesson covers advanced joins and database security to protect sensitive data from unauthorised access or manipulation. We will create tables for students, courses, and enrollments while learning to secure stored data.

Now, let us explore **Table Relationships** and learn how to secure a database. Although table relationships are not always mandatory, they become crucial when related data is spread across multiple tables. They help ensure data integrity by reducing redundancy and inconsistencies.

Types of Database Relationships

The type of relationship between any two tables in a database depends on the number of records in one table that correspond to those in another table. Primarily, three types of relationships can be established between two tables in a relational database:

- (i) One-to-One
- (ii) One-to-Many
- (iii) Many-to-Many

One-to-One Relationship

A one-to-one relationship occurs when a single record in one table is associated with exactly one record in another, and vice versa. This type of relationship is typically used to split data into separate tables for better organisation, security, or performance.

Student ID	Student Name	Class	Date of Birth
1005	Mampi	10	12/04/2005
1008	Neeru	7	15/02/2008
1010	Akira	8	16/06/2007
1015	Leo	12	15/11/2002
Student Details Table			
Result ID	Student ID	Marks	Grade
101	1005	94	A
102	1008	70	B
103	1015	85	A
Student Result Table			

For example, the 'Student Details' table stores basic information about students, and the 'Student Result' table stores their marks. Since each student can have only one result, every record in the Student Details table is linked to a corresponding record in the Student Result table. This is a one-to-one relationship.

One-to-Many Relationship

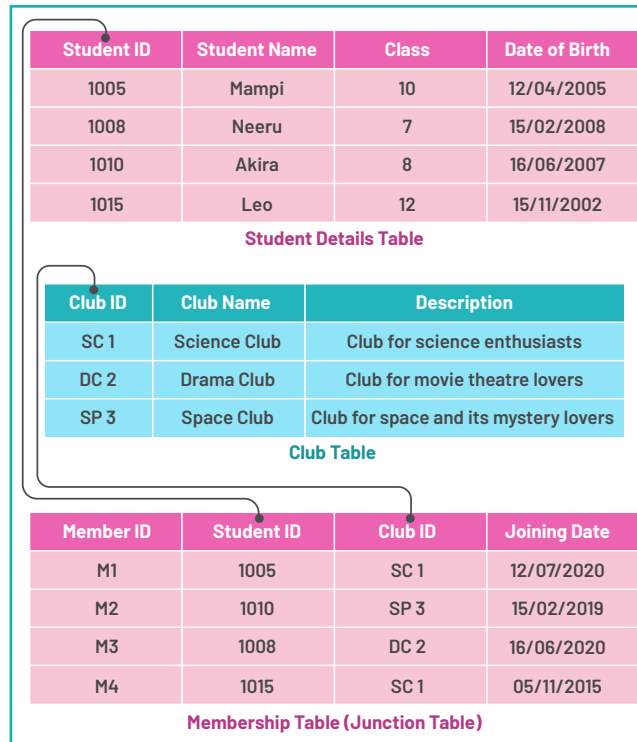
This is one of the most common types of database relationships. As the name suggests, a single record in one table is associated with multiple records in another table.

For example, consider an 'Authors' table that stores information about various authors and a 'Books' table containing details of different books. Since one author can write multiple books, each record in the Authors table can be linked to multiple records in the Books table. This is an example of a one-to-many relationship.

Author ID	Author Name	Date of Birth
1	Jane Austen	16/12/1775
2	Harper Lee	28/04/1926
3	Salman Rushdie	19/06/1947
Authors Table		
Book ID	Author ID	Book Name
101	1	Pride and Prejudice
102	2	To Kill a Mockingbird
103	2	Go Set a Watchman
104	3	Midnight's Children
105	3	Victory City
Books Table		

Many-to-Many Relationship

When multiple records in one table are associated with multiple records in another table, it is referred to as a many-to-many relationship. This type of relationship is typically implemented using a third table, known as a Junction or Bridge table, which links the records from the two tables.



For example, consider a 'Student Details' table that stores student details and a 'Club' table that records information about various clubs. A student can enroll in multiple clubs, and each club can have multiple students. This forms a many-to-many relationship.

Activity

We will design a simple Student-Course Enrollment system by creating a database that integrates information from multiple tables.

The database system will consist of three tables:

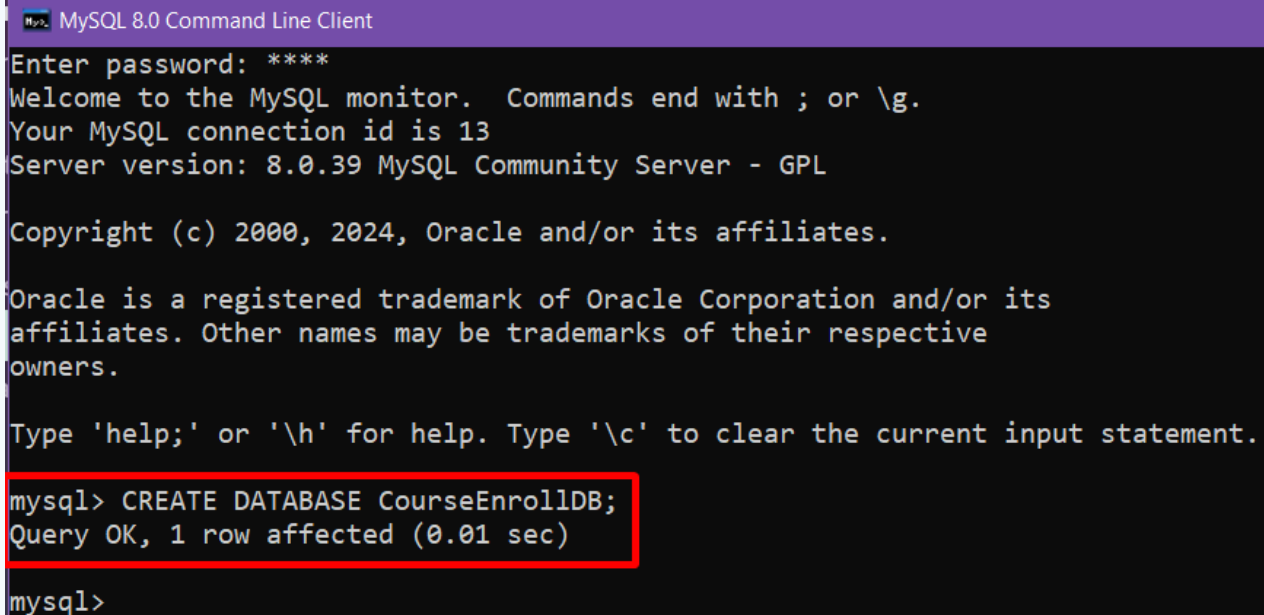
1. **Students Table:** Stores the basic details of all students.
2. **Courses Table:** Stores information about the different courses the school or college offers.
3. **Enrollments Table:** Stores details of course enrollments, specifying which student has opted for which course.

Activity

We will define the relationship between them using **foreign keys** and **joins**.

Part 1 - Create a Database

- 1 Open 'MySQL Client' and create a new database, **CourseEnrollDB** using the CREATE DATABASE command.



```
MySQL 8.0 Command Line Client
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 13
Server version: 8.0.39 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

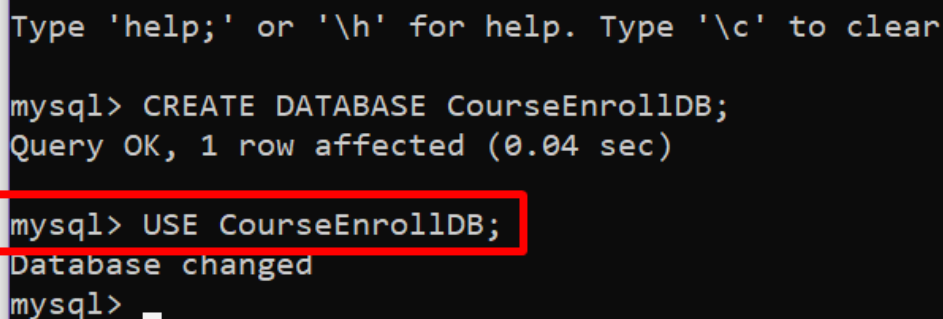
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE CourseEnrollDB;
Query OK, 1 row affected (0.01 sec)

mysql>
```

- 2 Enter the USE command and the database name to use the database.



```
Type 'help;' or '\h' for help. Type '\c' to clear

mysql> CREATE DATABASE CourseEnrollDB;
Query OK, 1 row affected (0.04 sec)

mysql> USE CourseEnrollDB;
Database changed
mysql> _
```

Activity

Part 2 - Create the 'Students' and 'Courses' tables

First, we will create the 'Students' table to store the students' basic details.

- 1 Create the 'Students' table by using the *CREATE TABLE* command in the MySQL Client. Include the following fields - 'student_id', 'student_name', 'student_email' and 'student_age'

 - Assign appropriate data types for each field and declare the 'student_id' field as the *PRIMARY KEY*.

```
sql> USE CourseEnrollDB;
Database changed
sql> CREATE TABLE Students (
  -> student_id INT PRIMARY KEY,
  -> student_name VARCHAR(100),
  -> student_email VARCHAR(100),
  -> student_age INT
  -> );
Query OK, 0 rows affected (0.24 sec)
```

- 2 Next, we will create a table 'Courses' to store details of various courses offered. Assign appropriate data types for each field and declare the 'course_id' field as the *PRIMARY KEY*.

```
mysql> CREATE TABLE Students (
  -> student_id INT PRIMARY KEY,
  -> student_name VARCHAR(100),
  -> student_email VARCHAR(100),
  -> student_age INT
  -> );
Query OK, 0 rows affected (0.24 sec)

mysql> CREATE TABLE Courses (
  -> course_id VARCHAR(50) PRIMARY KEY,
  -> course_name VARCHAR(100),
  -> course_description TEXT
  -> );
Query OK, 0 rows affected (0.07 sec)

mysql>
```

Part 3 - Store data on the Tables

- 1 In the MySQL Client, execute the *INSERT* command to create the first record in the 'Students' table. For the 'student_id' field, provide numerical values starting from 1.

 - Press 'Enter' once completed.

```
mysql> CREATE TABLE Courses (
  -> course_id VARCHAR(50) PRIMARY KEY,
  -> course_name VARCHAR(100),
  -> course_description TEXT
  -> );
Query OK, 0 rows affected (0.07 sec)

mysql> INSERT INTO Students(student_id, student_name, student_email, student_age)
  -> VALUES (1, 'Lily', 'lily2002@gmail.com', 22);
Query OK, 1 row affected (0.08 sec)
```

Activity

We can also enter multiple records using a single *INSERT* statement.

- 2 Add a few more student records, as shown below. Press the 'Enter' key after each record entry to move to the next line.

```
MySQL 8.0 Command Line Client
mysql>
mysql> INSERT INTO Students(student_id, student_name, student_email, student_age)
-> VALUES
-> (2, 'Lian', 'lian2006@gmail.com', 18),
-> (3, 'Meera', 'iammeera@gmail.com', 20),
-> (4, 'Alex', 'alex8929@gmail.com', 21),
-> (5, 'James', 'james@gmail.com', 22);
Query OK, 4 rows affected (0.02 sec)
Records: 4 Duplicates: 0 Warnings: 0
mysql>
```

Note:

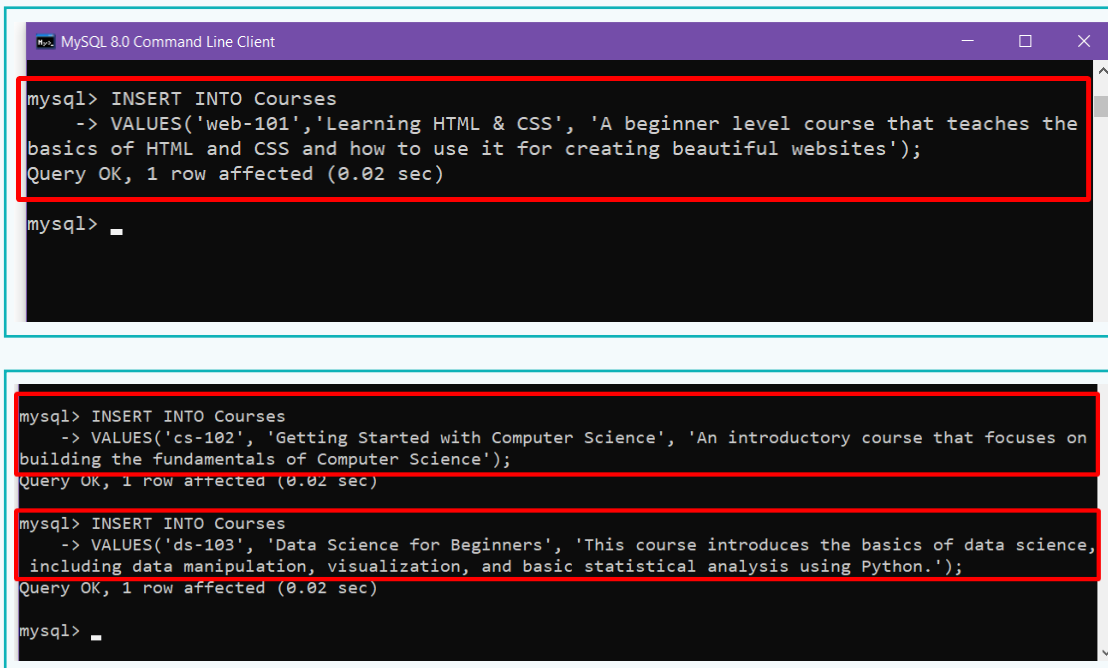
Separate each record with a comma and end the final record with a semicolon.

- 3 Verify that data has been successfully stored in the table using the *SELECT* command.

```
mysql> SELECT * FROM Students;
+-----+-----+-----+-----+
| student_id | student_name | student_email | student_age |
+-----+-----+-----+-----+
| 1 | Lily | lily2002@gmail.com | 22 |
| 2 | Lian | lian2006@gmail.com | 18 |
| 3 | Meera | iammeera@gmail.com | 20 |
| 4 | Alex | alex8929@gmail.com | 21 |
| 5 | James | james@gmail.com | 22 |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
mysql>
```

Activity

- 4 Similarly, insert records in the 'Courses' table.



```
mysql> INSERT INTO Courses
-> VALUES('web-101','Learning HTML & CSS', 'A beginner level course that teaches the
basics of HTML and CSS and how to use it for creating beautiful websites');
Query OK, 1 row affected (0.02 sec)

mysql> _
```



```
mysql> INSERT INTO Courses
-> VALUES('cs-102', 'Getting Started with Computer Science', 'An introductory course that focuses on
building the fundamentals of Computer Science');
Query OK, 1 row affected (0.02 sec)

mysql> INSERT INTO Courses
-> VALUES('ds-103', 'Data Science for Beginners', 'This course introduces the basics of data science,
including data manipulation, visualization, and basic statistical analysis using Python.');
```

We have created two tables – 'Students' and 'Courses', each storing data specific to its respective entity. However, determining which student has enrolled in which course (or multiple courses) requires establishing a relationship between these tables.

To achieve this, we will have to create a third table called 'Enrollments'. This table will link the 'Students' and 'Courses' tables using a **Foreign Key**.

Part 4 - Create an Enrollments Table with Foreign Key

Recall:

- The table containing the foreign key is known as the **Child Table**, while the table containing the referenced primary key is called the **Parent Table**.

Activity

- 1 Create the 'Enrollments' table with the *FOREIGN KEY* declaration.

In the 'Enrollments' table,

- **enroll_id** is declared as the **Primary Key**.
- The 'std_id' and 'crs_id' fields are defined as **Foreign Keys**.
- The 'std_id' foreign key references the Primary key ('student_id') in the 'Students' table.
- The 'crs_id' foreign key references the Primary key ('course_id') in the 'Courses' table.

```
mysql> CREATE TABLE Enrollments(  
-> enroll_id INT PRIMARY KEY,  
-> std_id INT,  
-> crs_id VARCHAR(50),  
-> enroll_date DATE,  
-> FOREIGN KEY (std_id) REFERENCES Students(student_id),  
-> FOREIGN KEY (crs_id) REFERENCES Courses(course_id)  
-> );  
Query OK, 0 rows affected (0.38 sec)  
  
mysql>
```

Part 5 - Insert Records in the Enrollments Table

The 'Enrollments' table is designed to store information about student course enrollments. It records which student has enrolled in which course, enabling the retrieval of details about the courses chosen by each student.

For instance, if a student named **Meera** enrolls in the **Learning HTML** and **CSS** course, a corresponding entry will be added to the 'Enrollments' table. This record will establish an association between the student and the course within the database.

- 1 Insert a record into the 'Enrollments' table. For the 'enroll_id' field, we will consider numerical values starting from 100. Press 'Enter' at the end of the command.

```
MySQL 8.0 Command Line Client  
  
mysql> INSERT INTO Enrollments(enroll_id, std_id, crs_id, enroll_date)  
VALUES(100, 3, 'web-101', '2024-04-02');  
Query OK, 1 row affected (0.00 sec)  
  
mysql> _
```

In the *INSERT* statement, the value '3' is provided for the 'std_id' field and 'web-101' for the 'crs_id' field, as the 'student_id' of Meera is '3' and the 'course_id' of 'Learning HTML and CSS' is 'web-101'.

Activity

- 2 Similarly, insert enrollment records for 'Alex' in 'Data Science for Beginners' and 'Lily' in the 'Learning HTML and CSS' courses, respectively.

```
mysql> INSERT INTO Enrollments(enroll_id, std_id, crs_id, enroll_date)
-> VALUES(101, 4, 'ds-103', '2024-03-02');
Query OK, 1 row affected (0.02 sec)

mysql> INSERT INTO Enrollments(enroll_id, std_id, crs_id, enroll_date)
-> VALUES(102, 1, 'web-101', '2024-07-02');
Query OK, 1 row affected (0.00 sec)

mysql>
```

In this way, every time a student enrolls in a course, the details will be stored in the 'Enrollments' table using the corresponding IDs.

- 3 View all records stored in the 'Enrollments' table using the SELECT statement.

```
MySQL 8.0 Command Line Client

mysql> SELECT * FROM Enrollments;
+-----+-----+-----+-----+
| enroll_id | std_id | crs_id | enroll_date |
+-----+-----+-----+-----+
|         100 |      3 | web-101 | 2024-04-02 |
|         101 |      4 | ds-103  | 2024-03-02 |
|         102 |      1 | web-101 | 2024-07-02 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> _
```

Now that all records have been stored let us explore how to retrieve data from the 'Students' and 'Courses' tables using the information in the 'Enrollments' table.

Activity

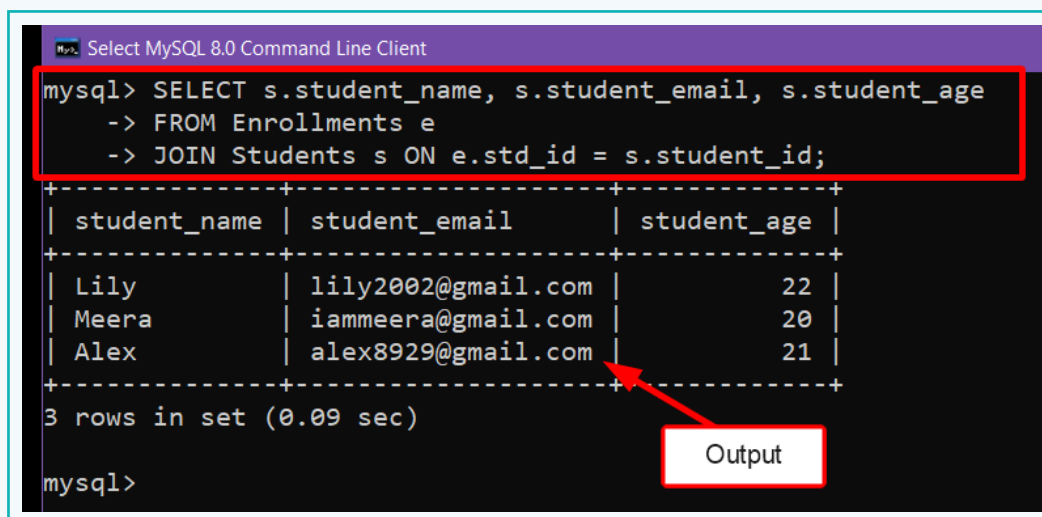
Part 6 - Retrieve Data using Joins

In order to retrieve data from multiple tables, we will implement Joins.

Recall:

Joins retrieve data from two or more tables by combining records through related columns, mainly using primary and foreign keys.

- 1 Retrieve records of only those students who have enrolled in any of the courses.



```
mysql> SELECT s.student_name, s.student_email, s.student_age
-> FROM Enrollments e
-> JOIN Students s ON e.std_id = s.student_id;
```

student_name	student_email	student_age
Lily	lily2002@gmail.com	22
Meera	iammeera@gmail.com	20
Alex	alex8929@gmail.com	21

3 rows in set (0.09 sec)

mysql>

Output

Here,

- **SELECT**

This clause specifies the columns to retrieve. 's' here acts as an alias to the 'Students' table for easier reference.

- a. **s.student_name**: The student's 'name' from the 'Students' table.
- b. **s.student_email**: The student's 'email' from the 'Students' table.
- c. **s.student_age**: The student's 'age' from the 'Students' table.

- **FROM Enrollments e**

This indicates that the query will start with the Enrollments table. The 'e' here is an alias to the 'Enrollments' table for easier reference.

Activity

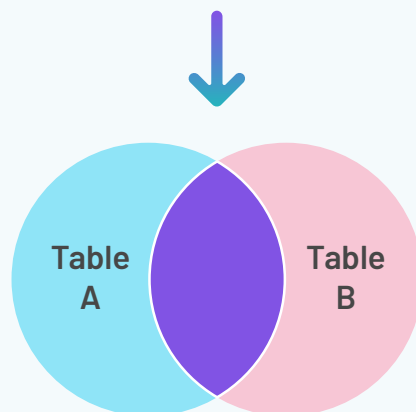
1 *JOIN Students s ON e.std_id = s.student_id*

This is an **Inner Join** between the 'Enrollments' and 'Students' tables, with the 'student_id' column serving as the common or linking column. 's' here acts as an alias to the 'Students' table.

Recall:

An **Inner Join** is a type of join that returns records with a matching value in both tables.

Student ID	Name	Student ID	Subject
101	Liam	104	Physics
102	Emma	105	Computer Science
103	Noah	106	Mathematics
104	Olivia	107	Geography
		108	Chemistry



Student ID	Name	Subject
104	Olivia	Physics

Activity

- 2 Update the above query to include the course enrollment date by specifying the 'enroll_date' column in the *SELECT* statement.

Note:

'enroll_date' needs to be prefixed with 'e', as this field belongs to the 'Enrollments' table.

```
mysql> SELECT s.student_name, s.student_email, s.student_age, e.enroll_date
-> FROM Enrollments e
-> JOIN Students s ON e.std_id = s.student_id;
```

student_name	student_email	student_age	enroll_date
Meera	iammeera@gmail.com	20	2024-04-02
Alex	alex8929@gmail.com	21	2024-03-02
Lily	lily2002@gmail.com	22	2024-07-02

3 rows in set (0.00 sec)

- 3 Retrieve the records of all the enrollments, including student name, student email, and course name.

```
MySQL 8.0 Command Line Client
mysql> SELECT s.student_name, s.student_email, c.course_name
-> FROM Enrollments e
-> JOIN Students s ON e.std_id = s.student_id
-> JOIN Courses c ON e.crs_id = c.course_id;
```

student_name	student_email	course_name
Lily	lily2002@gmail.com	Learning HTML & CSS
Meera	iammeera@gmail.com	Learning HTML & CSS
Alex	alex8929@gmail.com	Data Science for Beginners

3 rows in set (0.03 sec)

Output

This query is similar to the previous one and includes the course name (c.course_name) along with student details.

The statement **JOIN Courses c ON e.crs_id = c.course_id** forms an 'Inner Join' between the 'Enrollments' and 'Courses' tables using the 'course_id' as the common or linking column.

Now, we will retrieve the enrollment details of a particular student. Recall that the **WHERE** clause is used along with the *SELECT* statement to retrieve specific records based on a particular condition.

Activity

- 4 Use the WHERE clause to filter out the records in this case.

```
mysql> SELECT s.student_name, s.student_email, c.course_name, e.enroll_date
-> FROM Enrollments e
-> JOIN Students s ON e.std_id = s.student_id
-> JOIN Courses c ON e.crs_id = c.course_id
-> WHERE s.student_id = 3;
```

student_name	student_email	course_name	enroll_date
Meera	iammeera@gmail.com	Learning HTML & CSS	2024-04-02

1 row in set (0.01 sec)

Output

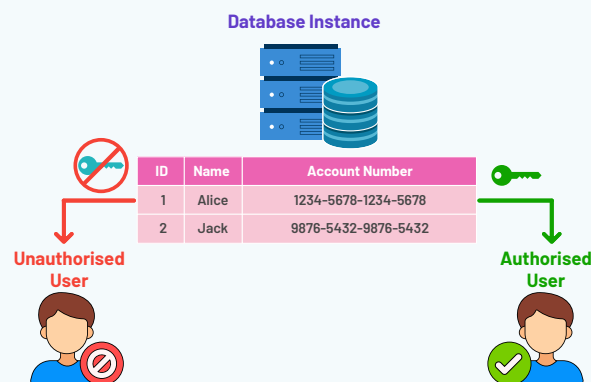
In this way, Joins allow us to handle various data queries efficiently.

Now that we know how to retrieve and combine data using Joins, it is equally important to protect the stored data. Databases often store sensitive information, such as personal credentials and private data, which must be protected from unauthorised access or breaches.

Let us explore key security measures to keep our data safe and accessible only to authorised users.

Database Security

Database security refers to the implementation of measures and practices to protect a database from unauthorised access, data breaches, and potential data loss. Ensuring database security is essential for maintaining data confidentiality, integrity, and availability.



Some key aspects of database security are:

- **Authentication:** Grants access only to authorised users, typically through a username and password.
- **Authorisation:** This can be considered restrictive access, defining which users can access or modify a database. For example, a certain user may have permission to create, delete, or update a database, whereas others may only view the data stored and not make any changes to it.

Activity

- **Encryption:** Sensitive data such as bank account or card numbers, PAN, etc., should be encrypted to enhance protection.
- **Backup:** A periodic backup system may help with data recovery in case of data loss, corruption, or cyberattack.
- **Firewalls:** Firewalls can be used to protect a database from malicious or unauthorised connections.

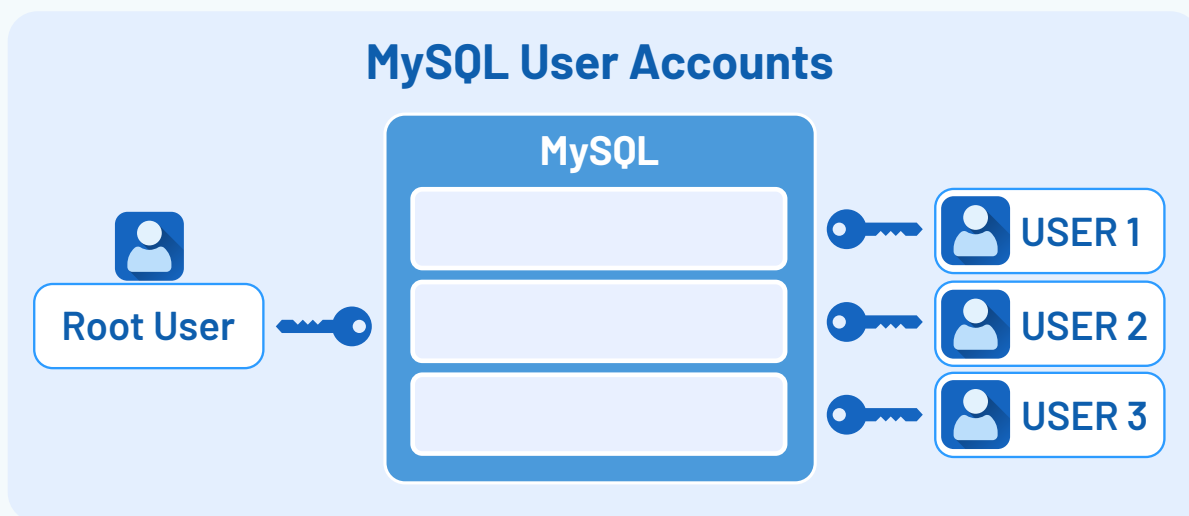
Now, let us implement authentication and authorisation to enhance database security. We will start by creating a new user and granting him/her restrictive access.

Part 7 – Create a New User

Just as we need a user account to log in to a computer, MySQL also requires a user account to access and manage databases. When MySQL is installed, a default user account called **root** is created. This **root user** controls all databases, tables, and user accounts.

However, if multiple people work on the same database, granting everyone access to the root account can lead to serious security risks.

Instead, separate user accounts with limited access can be created. Each user can be granted specific permissions based on their tasks, ensuring that they can only perform the necessary operations without compromising the database's security.



Activity

Syntax to create a new user:

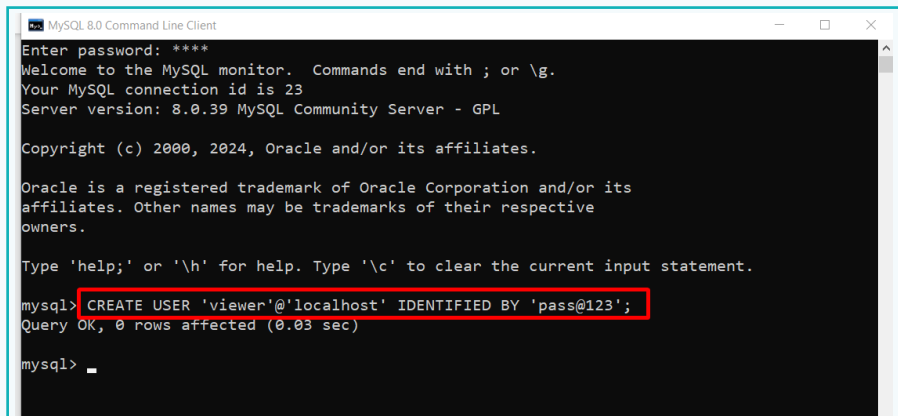
```
CREATE USER 'username'@'hostname' IDENTIFIED BY 'password';
```

Here,

- **CREATE USER** is the command to create a new user.
- **'username'** is the name of the user being created.
- **@'hostname'** specifies the host or server from which the user will connect to the database. If the user is accessing the database from their own computer, **'localhost'** can be used as the hostname.
- **IDENTIFIED BY 'password'** sets a password for the new user.

1 Create a new MySQL user.

- Open the MySQL client (if closed) and create a new user. You can choose your name here or a different one. We will go ahead with the name 'Viewer' for this activity. Use the **IDENTIFIED BY** keyword to set a password for the user.



```
MySQL 8.0 Command Line Client
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 23
Server version: 8.0.39 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

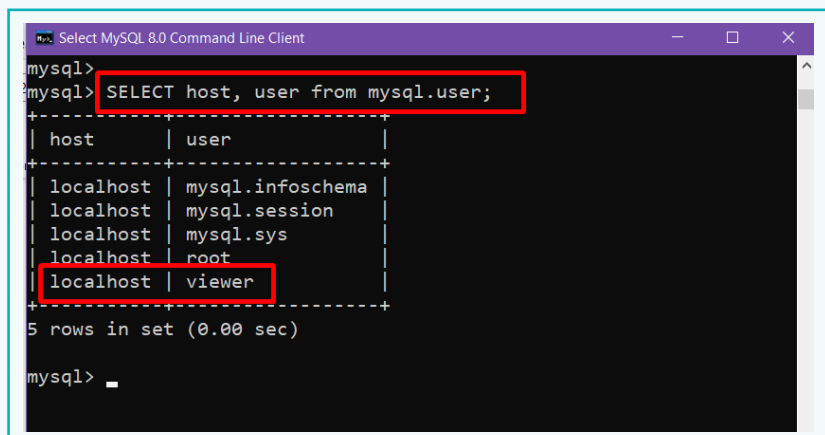
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE USER 'viewer'@'localhost' IDENTIFIED BY 'pass@123';
Query OK, 0 rows affected (0.03 sec)

mysql>
```

2 Verify that the user has been created by retrieving data from the **mysql.user** table using the **SELECT** command.

- **mysql.user** is a system table in MySQL that stores information about all users. It consists of two fields: *host* and *user*.



```
Select MySQL 8.0 Command Line Client
mysql> SELECT host, user from mysql.user;
+-----+-----+
| host      | user      |
+-----+-----+
| localhost | mysql.infoschema |
| localhost | mysql.session |
| localhost | mysql.sys   |
| localhost | root       |
| localhost | viewer    |
+-----+-----+
5 rows in set (0.00 sec)

mysql>
```

Notice the user 'viewer' is in the list, which confirms that the account has been successfully created.

Activity

The next step is to grant this user the required permissions, allowing it to perform specific database actions.

Part 8 – Grant User Permissions or Privileges

In MySQL, **Permissions** or **Privileges** define the rights granted to a user, determining the operations they are allowed to perform on a database.

Some of the common privileges are:

- **SELECT:** View data stored in a table.
- **INSERT:** Add new records to a table.
- **UPDATE:** Modify existing records in a table.
- **DELETE:** Remove records from a table.
- **CREATE:** Create new databases, tables, and other objects.

MySQL offers the *GRANT* command to grant permissions to a user.

Syntax:

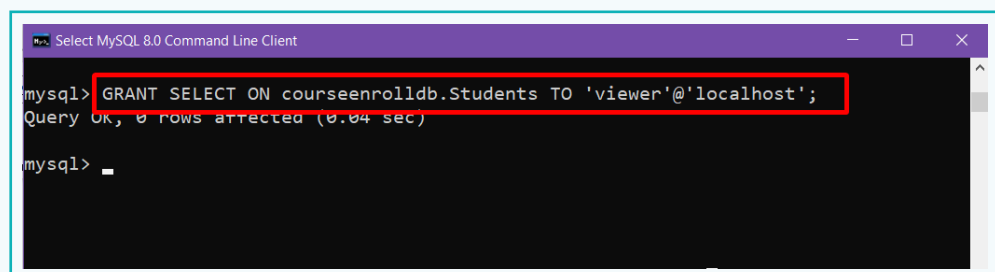
GRANT privilege ON database_name.table_name TO 'username'@'hostname';

Here,

- **privilege** refers to actions the user can perform, like SELECT, INSERT, CREATE, etc. A user can be granted multiple privileges by separating each with a comma.
- **database_name.table_name** specifies the database and table name.
- **'username'@'hostname'** specifies the user to whom the privilege is granted.

- 1 Write the *GRANT* command using the *SELECT* privilege to the newly created user 'viewer'.

This will allow the 'viewer' user only to read the data from the 'Students' table in the 'CourseEnrollDB' database.



```
Select MySQL 8.0 Command Line Client
mysql> GRANT SELECT ON courseenrolldb.Students TO 'viewer'@'localhost';
Query OK, 0 rows affected (0.04 sec)
mysql> _
```

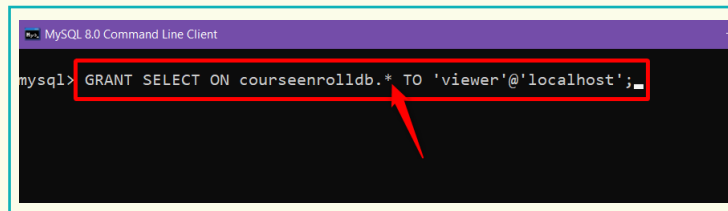
Note:

By default, SQL is not case-sensitive for keywords or identifiers in most databases.

Activity

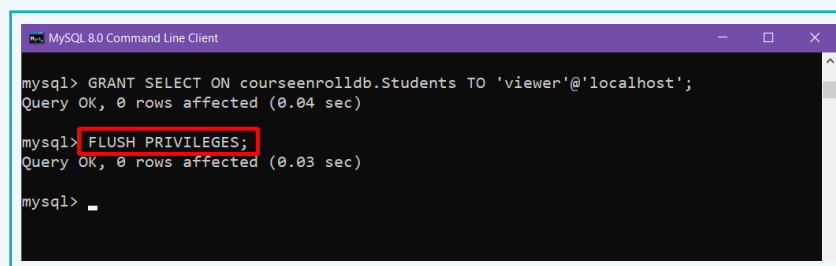
Note:

To grant the *SELECT* privilege on all the tables, replace the table 'Students' with an asterisk (*).



```
mysql> GRANT SELECT ON courseenrolldb.* TO 'viewer'@'localhost';
```

- 2 Next, we must execute the *FLUSH PRIVILEGES* command to apply the new privileges. This command reloads the privilege tables and is required whenever changes are made to user accounts or permissions.



```
mysql> GRANT SELECT ON courseenrolldb.Students TO 'viewer'@'localhost';
Query OK, 0 rows affected (0.04 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.03 sec)

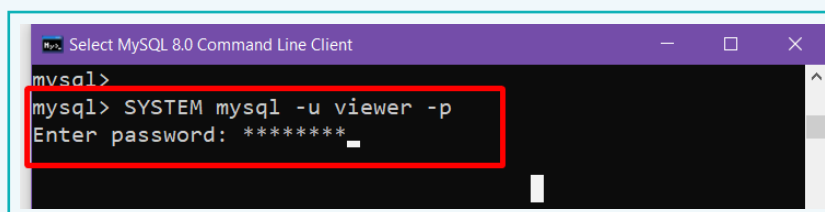
mysql>
```

Test the Granted Privilege

Now, let us test the privilege granted to the user 'viewer'. To do so, we must first log in to MySQL using the 'viewer' account instead of the 'root' user.

- 3 Run the following command to log into the MySQL client using the 'viewer' account. Specify the username 'viewer' using the **-u** flag, followed by the **-p** flag.

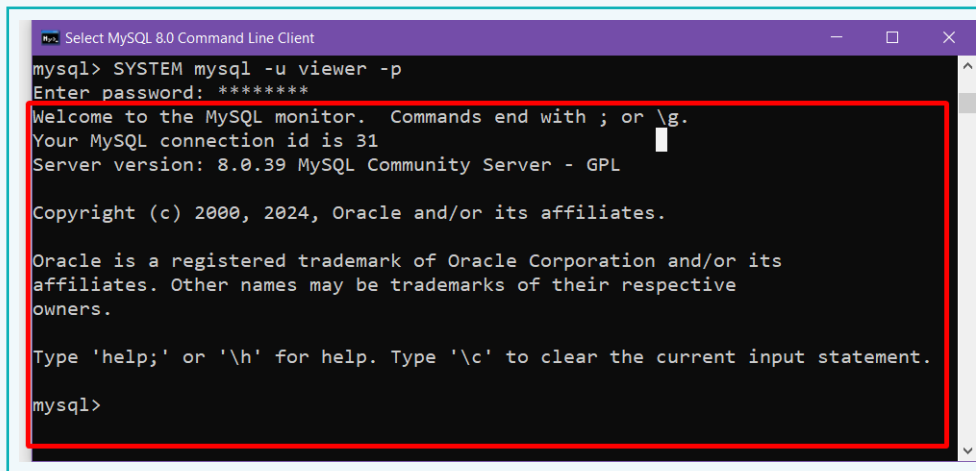
Note that you must press the 'Enter' key after the **-p** flag and specify the password in the following line. Press Enter again after providing the password.



```
mysql> SYSTEM mysql -u viewer -p
Enter password: *****
```

Activity

The following window will appear if login is successful.



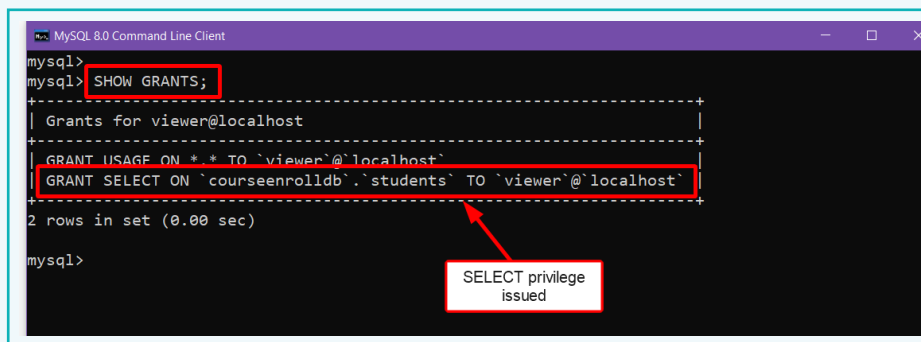
```
Select MySQL 8.0 Command Line Client
mysql> SYSTEM mysql -u viewer -p
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 31
Server version: 8.0.39 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql>
```

- 4 Use the *SHOW GRANTS* command to check the privileges issued to the 'viewer' account.

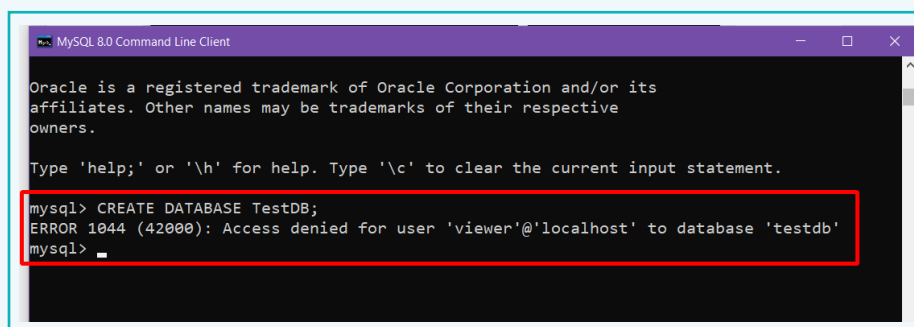


```
mysql> SHOW GRANTS;
+-----+
| Grants for viewer@localhost |
+-----+
| GRANT USAGE ON *.* TO `viewer`@`localhost` |
| GRANT SELECT ON `courseenrolldb`.`students` TO `viewer`@`localhost` |
+-----+
2 rows in set (0.00 sec)

mysql>
```

SELECT privilege issued

- 5 Now, we will try executing a simple *CREATE DATABASE* command to see if it is allowed.

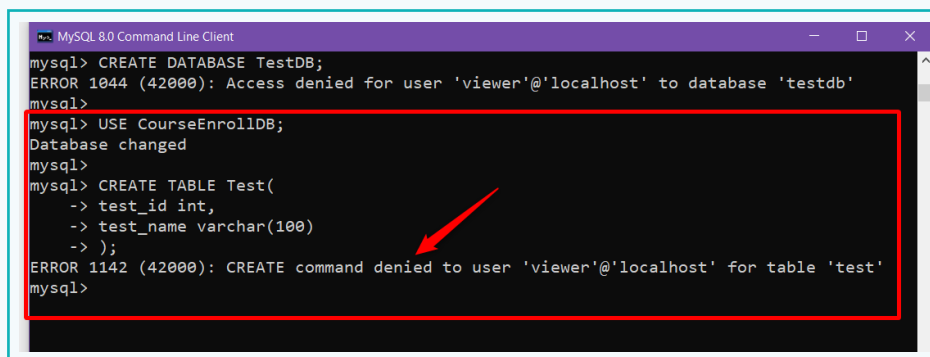


```
mysql> CREATE DATABASE TestDB;
ERROR 1044 (42000): Access denied for user 'viewer'@'localhost' to database 'testdb'
mysql>
```

Activity

We can see that an **'Access denied'** response is displayed, which prevents the user from creating a database.

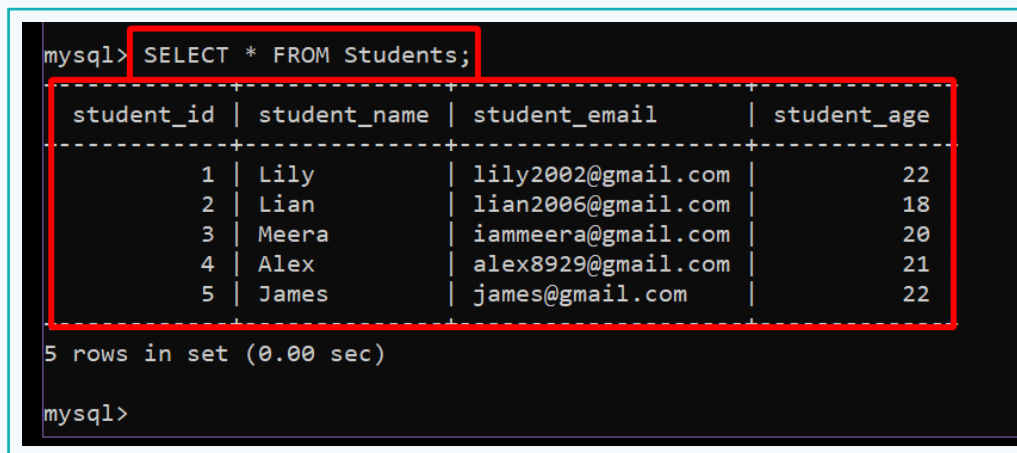
- 6 Next, we will check if we can create a new table in the 'CourseEnrollDB' database.



```
mysql> CREATE DATABASE TestDB;
ERROR 1044 (42000): Access denied for user 'viewer'@'localhost' to database 'testdb'
mysql>
mysql> USE CourseEnrollDB;
Database changed
mysql>
mysql> CREATE TABLE Test(
  -> test_id int,
  -> test_name varchar(100)
  -> );
ERROR 1142 (42000): CREATE command denied to user 'viewer'@'localhost' for table 'test'
mysql>
```

Again, we have been denied permission to create a table.

- 7 Since we have only granted the *SELECT* privilege to the user, let us verify its functionality.



```
mysql> SELECT * FROM Students;
```

student_id	student_name	student_email	student_age
1	Lily	lily2002@gmail.com	22
2	Lian	lian2006@gmail.com	18
3	Meera	iammeera@gmail.com	20
4	Alex	alex8929@gmail.com	21
5	James	james@gmail.com	22

```
5 rows in set (0.00 sec)

mysql>
```

We are able to retrieve data from the 'Students' table using the *SELECT* command. This implies that we can only read data from the table, not create or update anything.

Revoking User Privilege

There may be situations where users no longer require specific permissions, or you must restrict their access to maintain database security. In such cases, the *REVOKE* command is used to remove particular privileges from a user.

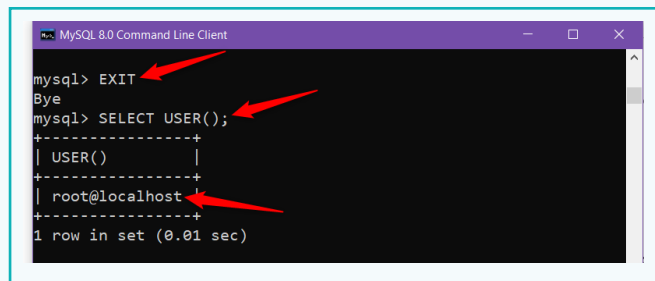
Activity

Syntax:

`REVOKE privilege ON database_name.table_name FROM 'username'@'hostname';`

Let us revoke the *SELECT* privilege from the 'viewer' user.

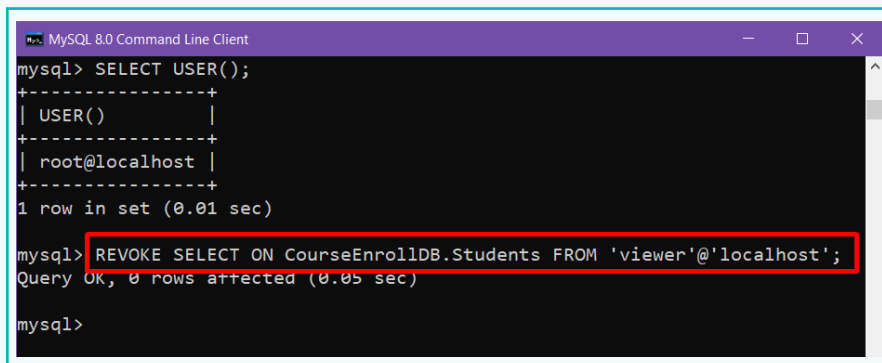
- 8 First, we must log out of the viewer account using the *EXIT* command. We can also execute the *SELECT USER()* command to check the currently logged-in user.



```
mysql> EXIT
Bye
mysql> SELECT USER();
+-----+
| USER() |
+-----+
| root@localhost |
+-----+
1 row in set (0.01 sec)
```

We can observe that we are currently logged out of the 'viewer' account and logged in as the **root** user.

- 9 Now, let us execute the *REVOKE* command.

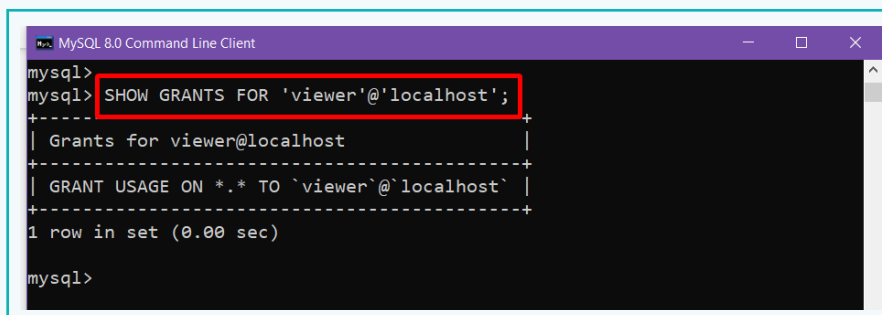


```
mysql> SELECT USER();
+-----+
| USER() |
+-----+
| root@localhost |
+-----+
1 row in set (0.01 sec)

mysql> REVOKE SELECT ON CourseEnrollDB.Students FROM 'viewer'@'localhost';
Query OK, 0 rows affected (0.05 sec)

mysql>
```

- 10 We can use the *SHOW GRANTS* command, as shown, to confirm that the *SELECT* privilege is revoked from the viewer account.



```
mysql> SHOW GRANTS FOR 'viewer'@'localhost';
+-----+
| Grants for viewer@localhost |
+-----+
| GRANT USAGE ON *.* TO `viewer`@`localhost` |
+-----+
1 row in set (0.00 sec)

mysql>
```

Activity

We can observe that the `SELECT` privilege is no longer granted to the 'viewer' account. The **GRANT USAGE ON *.* TO `viewer`@`localhost`** output implies that no privileges are given to that user. It just allows a user to authenticate and connect to the MySQL server.

In this lesson, we designed a Student-Course Enrollment system using multiple tables linked through foreign keys. We learnt to retrieve data using Joins and explored Database Security by creating users, granting specific privileges, and revoking them to ensure authorised access and data protection.

Exercise

Fill in the Blanks.

- 1 _____ are used to retrieve data from more than one table.
- 2 The _____ command is used to create a new user.
- 3 To set a password for a newly created user, the _____ keyword is used.
- 4 The _____ is a system table in MySQL that stores information about all the users.
- 5 To put new privileges into effect, the _____ command is used.

Exercise

 **Tick Mark the Correct Answer.**

- 6** Which keyword is used when creating a Foreign Key?
- a. References
 - b. Create
 - c. Primary Key
 - d. Alter

 **State if the Statements are True or False.**

- 7** Numeric values are placed within single (' ') or double quotes (" ").

 _____

- 8** The INSERT privilege allows a user to view data stored in a database.

 _____

- 9** MySQL offers the GRANT command to grant permissions to a user.

 _____

- 10** We can use the DISPLAY GRANTS command to check the privileges of a particular user.

 _____

 **Answer in One Line.**

- 11** What is the syntax to revoke granted privileges from a user?

 _____

Tech Flash

DBMS or Database Management System : A software package that enables users to create, maintain, and use a database.

Privilege : A privilege in a database is a specific permission granted to a user, allowing them to perform actions like 'SELECT', 'INSERT', 'UPDATE', 'DELETE', 'CREATE', or 'DROP' on a database, table, or column.